PATRONES DE DISEÑO · BACKEND + FRONTEND

Plataforma de Comercio de Energía

12 patrones aplicados a un mercado real de excedentes

Laravel + PHP · React + TypeScript · 26 implementaciones

12 patrones × 2 capas

El proyecto en 30 segundos

PROBLEMA

Los excedentes de pequeños productores se pierden: no hay un canal simple para publicarlos, compararlos y venderlos.

SOLUCIÓN

Un mercado virtual para publicar excedentes, comprar energía, subastar, integrar IoT y predecir producción y consumo.

ALCANCE

Prototipo académico: sin pagos ni hardware real. El estado se persiste en archivos JSON.

Un prototipo funcional que conecta producción, consumo y predicción.

Usuarios y subsistemas

USUARIOS

- **Productor —** genera energía y vende sus excedentes.
- **Consumidor —** compra la energía disponible.
- **Doble rol —** puede producir y consumir a la vez.

CUATRO SUBSISTEMAS

- **1. Compra y venta —** ofertas, compras e historial.
- **2. Subastas —** mejor oferta en tiempo real.
- **3. IoT —** medidores e inversores simulados.
- **4. Predicción —** producción y consumo.

CÓMO FLUYE LA ENERGÍA

> Un mismo usuario puede ser productor y consumidor: el mercado conecta ambos roles.

Dos capas, los mismos patrones

FRONTEND · React + TypeScript

10 paneles + patrones del navegador

▼ HTTP / JSON

BACKEND · Laravel + PHP

EnergyController + patrones del servidor

▼ lectura / escritura

PERSISTENCIA · JSON en storage/app

market_data · logger_data · flyweight_data · proxy_cache

Cada capa resuelve el mismo problema en su lenguaje.

Los 12 patrones

CREACIONALES

Cómo se crean los objetos

- Singleton
- Factory Method
- Builder
- Prototype

ESTRUCTURALES

Cómo se componen los objetos

- Adapter
- Bridge
- Decorator
- Composite
- Facade
- **Flyweight · NUEVO**
- **Proxy · NUEVO**

COMPORTAMIENTO

Cómo colaboran los objetos

- Abstract / Template Method

26 implementaciones documentadas con UML, código, endpoints y demo.

Singleton

> “Una sola instancia y un punto de acceso global.”

PROBLEMA

- El mercado, los logs y la configuración no pueden estar duplicados.

EN EL PROYECTO

- MarketService, LoggerService y ConfigService (PHP).
- EnergyApiService: un único cliente HTTP (TS).
- **Archivos:** MarketService.php · EnergyApiService.ts

Demo · Todos los paneles comparten el mismo estado y el mismo cliente.

Factory Method

> “Recibe un tipo y devuelve la clase concreta correcta.”

PROBLEMA

- Crear objetos según el tipo sin llenar el código de “new”.

EN EL PROYECTO

- EnergySourceFactory: solar / wind / hydro.
- PredictorFactory y EnergyCardFactory.
- **Archivos:** EnergySourceFactory.php · EnergyCardFactory.tsx

Demo · Catálogo de fuentes y tarjetas de energía; un tipo inválido responde 422.

Abstract / Template Method

> “El esqueleto del algoritmo vive en la base; las subclases cambian los datos.”

PROBLEMA

- La predicción diaria y la semanal comparten toda la lógica.

EN EL PROYECTO

- AbstractPredictor::predict() define los pasos.
- DailyPredictor y WeeklyPredictor solo aportan los datos.
- **Archivos:** AbstractPredictor.php · Predictor.ts

Demo · PredictionPanel compara el cálculo local (TS) y el del backend (PHP).

Builder

> “Construye un objeto complejo paso a paso.”

PROBLEMA

- Publicar una oferta implica muchos campos y variantes.

EN EL PROYECTO

- EnergyOfferBuilder expone los pasos encadenables.
- OfferDirector: oferta personalizada y oferta exprés.
- **Archivos:** EnergyOfferBuilder.php · EnergyOfferBuilder.ts

Demo · Asistente de 6 pasos en CreateOffer; POST /offers y /offers/express.

Prototype

> “Un objeto se copia a sí mismo y entrega un clon independiente.”

PROBLEMA

- Se publican ofertas parecidas una y otra vez.

EN EL PROYECTO

- OfferPrototype::copy() reinicia id, stock y fecha.
- PrototypeRegistry guarda plantillas por tipo de energía.
- **Archivos:** OfferPrototype.php · PrototypeRegistry.ts

Demo · Duplicar oferta: el clon no altera el original ni la plantilla.

Adapter

> “Traduce una interfaz incompatible a la que el cliente espera.”

PROBLEMA

- Medidores SCADA (texto) e inversores PV (JSON) hablan distinto.

EN EL PROYECTO

- DeviceGateway es la interfaz común.
- LegacyScadaAdapter y PvInverterAdapter traducen cada formato.
- **Archivos:** DeviceGateway.php · DeviceGateway.ts

Demo · DevicePanel muestra los datos crudos y los normalizados lado a lado.

Bridge

> “Separa abstracción e implementación para que varíen por separado.”

PROBLEMA

- Perfil (residencial/comercial) × moneda (COP/USD/EUR) = 6 clases.

EN EL PROYECTO

- EnergyBill: ResidentialBill y CommercialBill.
- CurrencyBridge: CopCurrency, UsdCurrency y EurCurrency.
- **Archivos:** EnergyBill.php · CurrencyBridge.ts

Demo · Matriz 2×3 de liquidaciones; añadir una moneda no toca los perfiles.

Decorator

> “Añade comportamiento envolviendo el objeto, sin una clase por combinación.”

PROBLEMA

- Tres extras generan 8 combinaciones posibles.

EN EL PROYECTO

- BasicOffer + Green (+5%), Insurance (+3%), Priority (+$8.000).
- OfferPricingService envuelve según los extras elegidos.
- **Archivos:** OfferDecorator.php · OfferDecorator.ts

Demo · OfferExtrasPanel recompone el total y lo compara con el backend.

Composite

> “Trata hojas y contenedores por igual; los totales se calculan recursivamente.”

PROBLEMA

- Un paquete puede contener ofertas u otros paquetes.

EN EL PROYECTO

- BundleComponent: interfaz común.
- OfferLeaf (hoja) y EnergyBundle (compuesto).
- **Archivos:** EnergyBundle.php · EnergyBundle.ts

Demo · BundlePanel añade ofertas y recalcula el total de todo el árbol.

Facade

> “Una interfaz simple para coordinar muchos subsistemas.”

PROBLEMA

- El cliente tendría que conocer y coordinar ocho servicios.

EN EL PROYECTO

- EnergyPlatformFacade: dashboard, publicar, comprar, asesoría, IoT.
- PlatformFacade delega desde el frontend.
- **Archivos:** EnergyPlatformFacade.php · PlatformFacade.ts

Demo · Una sola llamada a /platform/dashboard trae todo el panel.

Flyweight

> “Comparte lo que se repite; deja fuera lo único de cada objeto.”

PROBLEMA

- Cada oferta solar repite nombre, color y eficiencia del tipo.

EN EL PROYECTO

- Intrínseco: descriptor del tipo de energía (3 instancias).
- Extrínseco: productor, kWh y precio, en cada oferta.
- **Archivos:** EnergySourceFlyweightFactory.php · EnergyTypeFlyweightFactory.ts

Demo · Tres descriptores compartidos para todas las ofertas; /flyweight-stats.

Proxy

> “Mismo contrato que el objeto real, pero controla el acceso.”

PROBLEMA

- Consultar el mercado es costoso y las compras necesitan reglas.

EN EL PROYECTO

- Virtual: crea MarketService solo cuando se necesita.
- Caché TTL 10 s + protección de compras de 1 a 100 kWh.
- **Archivos:** MarketProxy.php · EnergyApiProxy.ts

Demo · Dos lecturas activan la caché; una compra de 999 kWh se bloquea.

Recorrido por la aplicación

1

Publicar oferta

Factory Method + Builder

Crea una oferta real desde el asistente.

2

Duplicar / plantilla

Prototype

Parte de una oferta o plantilla existente.

3

Predecir

Abstract / Template Method

Compara el cálculo TS con el PHP.

4

Dispositivos y liquidar

Adapter + Bridge

Unifica dispositivos y emite la liquidación.

5

Extras y paquetes

Decorator + Composite

Añade extras y agrupa ofertas.

6

Panel y métricas

Facade + Flyweight + Proxy

Panel único, métricas y bloqueo de compras.

Cada patrón se demuestra en la app, no solo en el código.

GRACIAS

12 patrones.
2 capas.
1 proyecto.

- Cada patrón resuelve un problema real del proyecto.
- Backend y frontend, con UML, código y endpoints.
- Flyweight y Proxy: las implementaciones nuevas.
- Demo funcional de punta a punta.

¿Preguntas?

Plataforma de Comercio de Energía